University of Essex Online

Assignment Title: **Design of a Banking Application with Focus on Safety and Cybersecurity**

Module: **Advanced Object-Oriented Design and Programming January 2026**

Course/Programme: **Online Professional Course: PG Computing**

University ID: **12702970**

e-portfolio: https://mowoh.github.io/.me/

Word count: **3629**

Declaration:

I hereby certify that this is my own original work. I have used any relevant technologies according to the assignment guidelines set out on Moodle.

Signature:

Date: 09.04.2026

Table of Contents

1 Introduction 3
1.1 The GUI 4
1.2 The Transaction Class 5
1.3 The Accounts Class 5
2 Design and Method 6
2.1 Design Goal 6
2.2 Design Challenges 7
2.3 The Architecture 7
2.3.1 Presentation Layer 7
2.3.2 Domain Layer 9
2.3.3 Authentication Layer 13
2.3.4 Persistence Layer 14
2.4 Main Application Interaction Flow 16
2.5 Summary of Design Patterns 18
2.6 Unit Tests 19
2.6.1 Test Summary 19
3 Academic Skill 21
3.1 Key Learnings during Secure Banking App Development 21
3.2 Summary of e-portfolio 24
4 Professional Development Plan 26
5 Conclusion 27
References 28

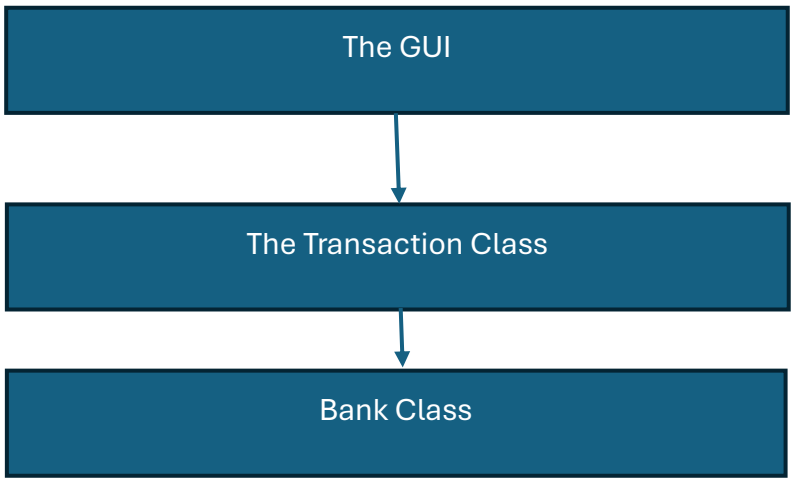
1 Introduction

Security is always at the forefront of every financial system's design consideration since such systems are a prime target for cyber-attacks. This project stresses the need to be even more intentional about applying data security principles such as multi-level encryption and decryption in communicating transaction information. The project starts off by employing modular, reusable and thread safe design techniques in its design of a basic banking application called Secure Banking App.

The Secure Banking App is based on the .Net framework 8.0 and written with the C# programming language. Being very small in size, with its solely educational purpose, Secure Banking App uses the layered-monolithic design architecture. Following this approach, the application is conceptually partitioned into three layers which share the business logic, each connecting via interfaces as shown in Figure1 below.

As the focus of the project is centered around cyber security and secure coding, the application includes encryption and description, as well as password protection features, with the former specifically designed to be used in the transaction class for performing safe and secure transactions. There is also a database class using SQLite [10] to demonstrate online transactions.

Figure 1: Fundamental component of the Secure Banking App

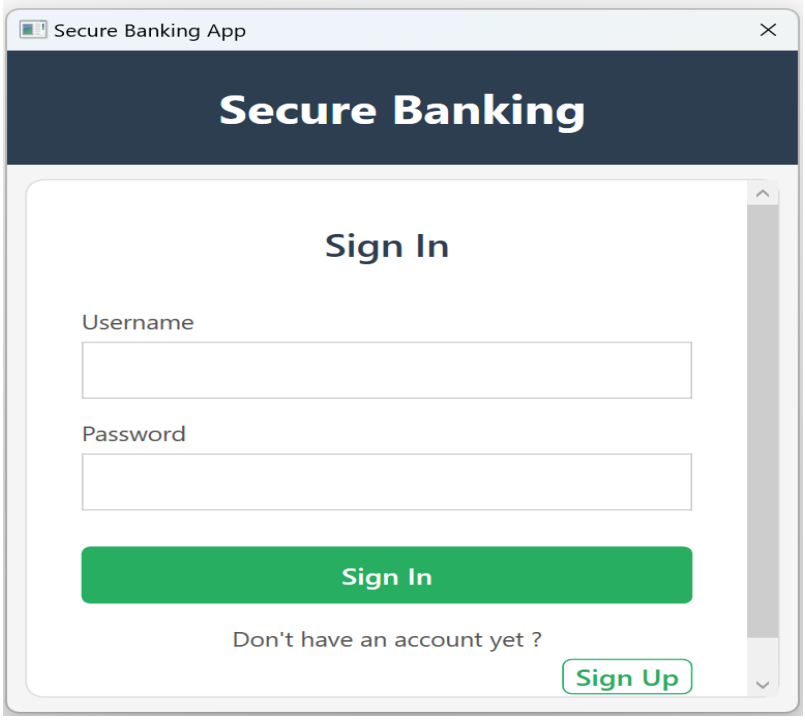


1.1 The GUI

The GUI for this application, whose design is discussed in the Design section is meant to give users the opportunity to interact with the application. Having a dimension of 450 by 550 mm, with a side scroller, it is aimed to give the user a mobile app feel, and offers basically logging in to the application, signing in, adding a bank account and making a transaction. The only transaction enabled is transfers between two accounts.

At launch, the application firstly loads the login page that allows the user to login either with his previously registered details or create a new login detail with the "Sign Up" button as show in Figure 2. A successful sign-in welcomes the user directly to the dashboard, where he can add accounts with initial deposits as well as view his accounts and balances

Figure 2: The login page of Secure Banking App



1.2 The Transaction Class

The transaction class is the middle layer between the low-level Bank Account and the GUI. It connects directly to the Bank Account, IAccount via defined interface shown in Snippet 1 below building on it to enable thread-safe withdrawals and deposits from the Bank Accounts. It also encapsulates important account security keys and initialization vector meant for secure transactions. Refer to Code Snippet 2 for the transaction class

Snippet 1: the IAccount interface

```
namespace SecureBankingApp.Accounts
{
    72 references
    public interface IAccount
    {
        23 references | 5/5 passing
        string Name { get; }
        14 references | 3/3 passing
        decimal Balance { get; }
        6 references | 3/3 passing
        void Deposit(decimal amount);
        5 references | 2/2 passing
        void Withdraw(decimal amount);
        4 references | 2/2 passing
        void WithdrawFromSavings(decimal amount);
        16 references | 8/8 passing
        void Save(int amount, TimeSpan duration);
    }
}
```

1.3 The Accounts Class

Two types of accounts are designed in this project viz: the Current account and the Savings Account. Both account types implement the general IAccount interface in Snippet 1 above through an inheritance hierarchy where the base class, AccountsBase, becomes the concrete class to the interface and an abstract class to a few methods like the Save(...) and GetAccountType(). There are shared methods like Withdraw() and Deposit() in the base class as well, which are only implemented once and shared across hierarchies.

The next section discusses the design concept of the whole application alongside this class in detail, by grouping them as different layers from the presentation layer through to the persistence layer.

2 Design and Method

The SecureBankingApp is a WPF-based [9] desktop banking application targeting .NET 8 and written in C# 12. Its purpose is to provide a simple banking workflow for user registration, login, account creation, dashboard viewing, and secure money transfer between accounts.

The application combines:

I. a presentation layer built with WPF pages and user controls,
II. a domain layer for accounts and transactions,
III. a persistence layer using SQLite,
IV. a security utility layer for hashing and encryption.

The design aims at separating concerns with the use of interfaces such as IAccount, IDatabase, ILoginBasic, and ILoginAdvanced.

2.1 Design Goal

The following are the design goals aimed at achieving with the Secure Banking App:

- provide a simple banking UI,
- support multiple account types such as current accounts, savings accounts, etc.
- persist account data
- support secure transactions and storage via the use of encryption
- encapsulate account behavior behind interfaces
- promote mocking and testability of the Accounts classes via the use of sufficient interfaces
- allow future extensibility through factories and abstractions
- employment of the test-driven development approach
- ensure thread safety using locks

2.2 Design Challenges

The challenges faced during the design of the Secure Banking App were the missing software architecture and UI design experience. The latter was overcome by long hours of studying WPF with xaml. The former was learned from the training modules, especially unit 9 as a skill and strengthened in the development process of this project. However, time is still a limiting factor, for getting the application to a production-ready state.

2.3 The Architecture

The Secure Banking App, being a layered monolithic architecture, comprises the following main layers

2.3.1 Presentation Layer

This layer provides the possibility for the program to interact with the user through the UI. The user can create an account and login as well as make transactions and view his account balances. The layer engulfs the following pages:

MainWindow:

The MainWindow acts as the application controller and navigation host. It is the central coordinator for login, sign-up, dashboard loading, account creation, transaction initiation, transaction confirmation.

This class currently mixes UI orchestration and business logic. The total separation of concerns for this class is planned as further development.

It stores application states such as current user, current and savings accounts, account numbers, registered accounts, database instance making it the de facto mediator of the application.

Dashboard:

The Dashboard is a page that displays username, savings balance, current balance, account numbers, recent transactions placeholder, buttons for logout, add account, and make transactions.

It uses DependencyProperty values for binding, which is appropriate for WPF UI updating.

Transaction:

The transaction presentation layer written in Transaction.xaml.cs is a UI page for entering source account type, destination account number, destination account name, amount. It raises events back to MainWindow:

- ConfirmButtonClickedEvent
- AccountNumberFormFilledClickedEvent
- CancelButtonClickedEvent

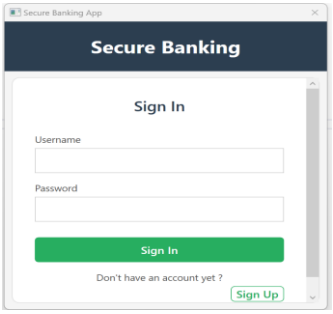
This is an event-driven interaction pattern.

SignUpPage and AccountRegPage:

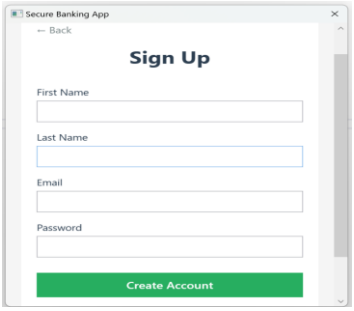
These user controls collect personal registration data, account type, initial deposit. They expose events instead of directly performing business operations, which is a good separation choice.

Figure 3 presents a typical usage flow for a new user, who does not have an already existing account and chooses to explore all the features of Secure Banking App like, create login details, login with the app-generated username and password, view the dashboard, add current or savings accounts, perform a transaction from one account to the other and logout.

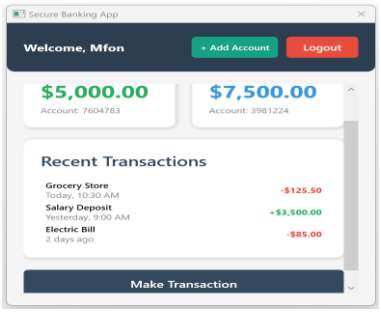
Figure 3: gallery of the presentation layer pages showing a typic usage flow



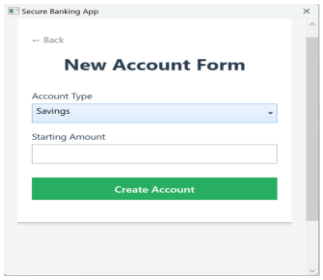
1. Login page



2. Signup page



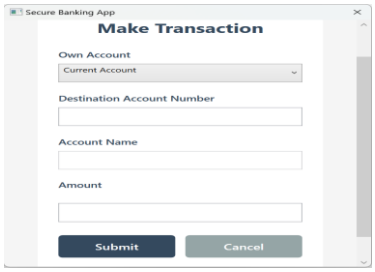
3. Dashboard



4. Account reg page

Direction of flow

1→2→1→3→4→3→5→3→1



5. Transaction page

2.3.2 Domain Layer

This layer holds the business logic of the Secure Banking App. As shown in Code Snippet 1, this layer is built as an abstraction using an interface. This interface defines the essential banking operations and allows multiple account implementations.

It aligns with the need for the Secure Banking App to follow the SOLID principle and allow for easy future extension without modifications.

The encapsulations as well as interaction among the different account types with base class and class interface is shown in the account hierarchy UML [11] in Figure 4. Notice private attributes, properties methods are preceded by a "– ", public by a "+" and shared by a "#".

AccountsBase:

AccountsBase provides shared behavior like account metadata storage, balance tracking, savings tracking, thread-safe deposit and withdrawal, and abstract savings behavior. This is a classic inheritance-based reuse design according to [8].

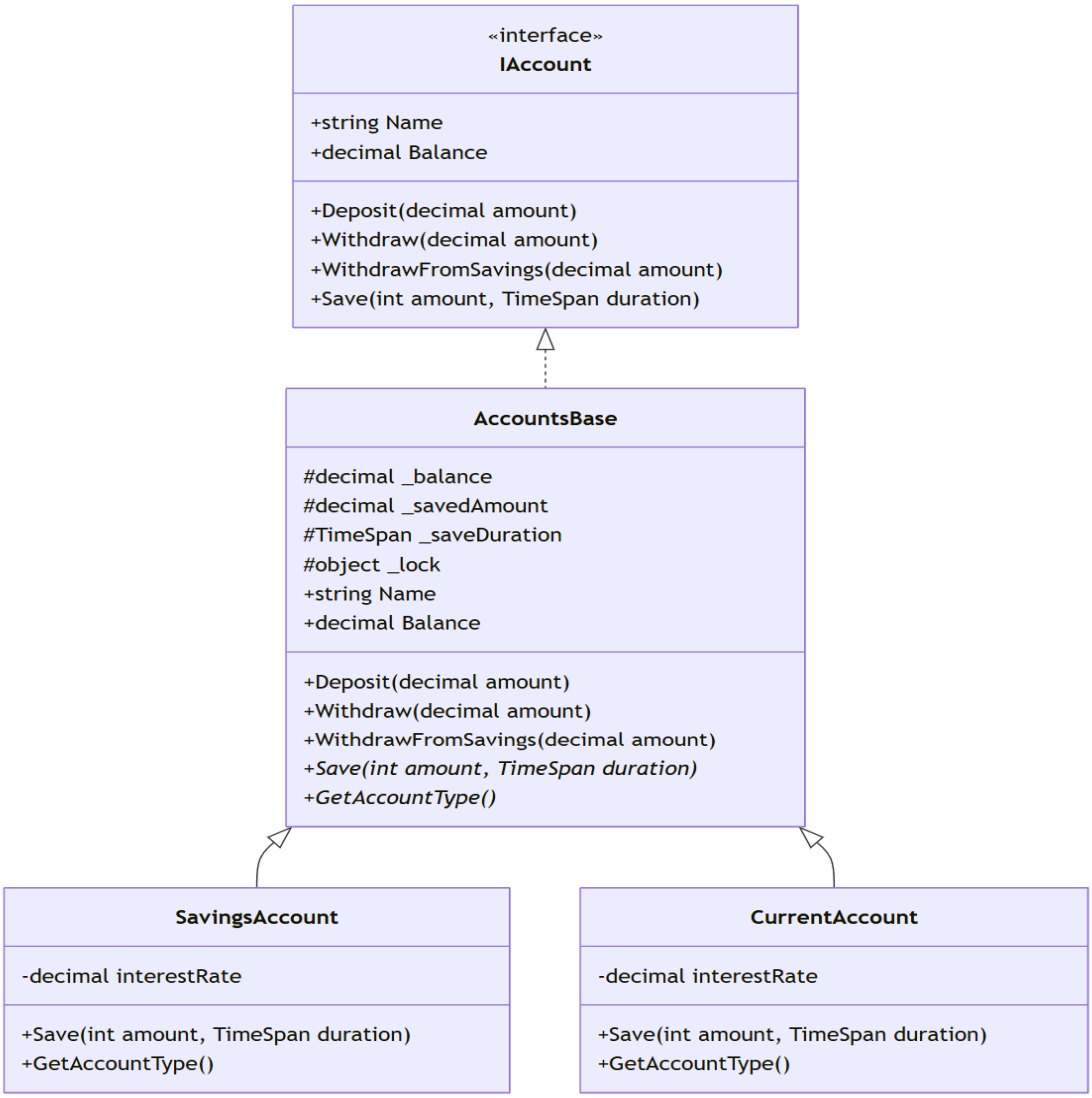
SavingsAccount:

SavingsAccount extends AccountsBase and applies a fixed interest rate of 0.05m, savings logic based on duration.

CurrentAccount:

CurrentAccount extends AccountsBase and applies a fixed interest rate of 0.02m, and a minimum savings duration of 30 days, implementing Save() a little differently.

Figure 4: Account hierarchy UML



Transactions.Transaction:

The domain transaction class is SecureBankingApp.Transactions.Transaction.

This layer has the following responsibilities:

- transfer money between accounts,
- save funds,
- withdraw from savings,
- securely send transaction data,
- securely receive transaction data.

Important Design Choices

- uses IAccount abstraction,
- uses lock ordering [7] to reduce deadlock risk,
- uses AES (Advanced Encryption System) [2] encryption through Crypt,
- stores encrypted transaction payloads in IDatabase.
- Follows the dependency injection approach, where the IAccount objects are passed in as constructor arguments as shown in Code Snippet 2 below

Code Snippet 2: showing the application of dependency injection in Transaction class

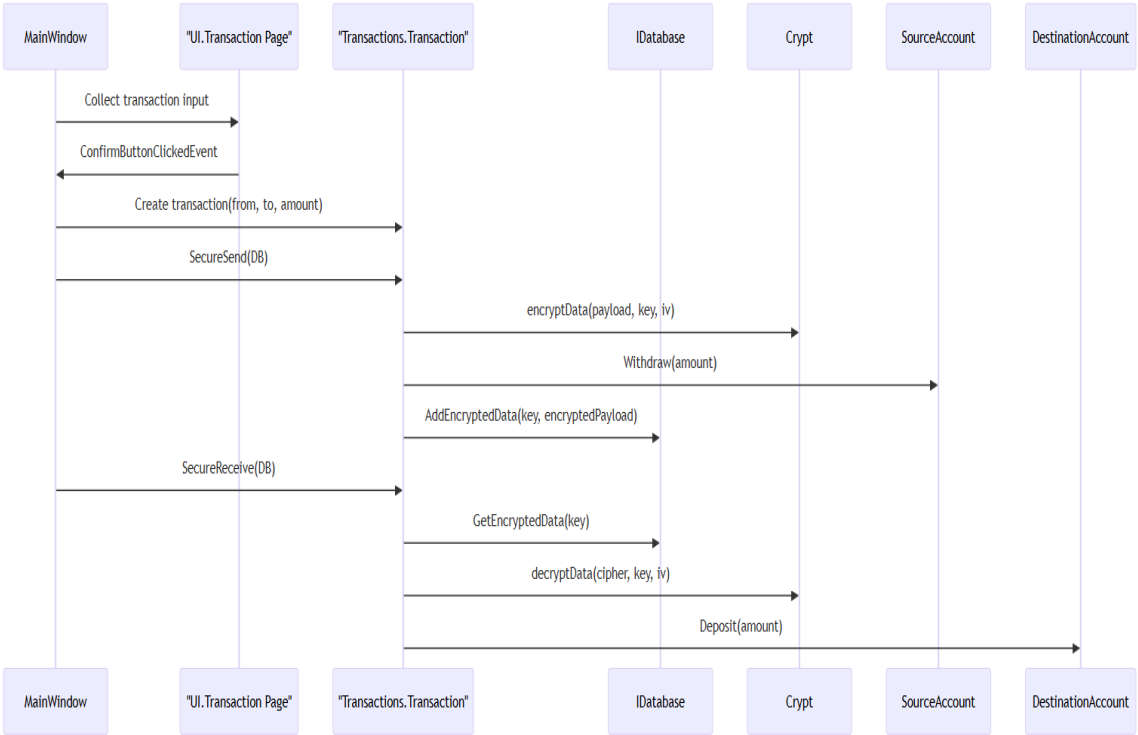
```
public class Transaction
{
  private IAccount _ownAccount;
  private IAccount _destinationAccount;
  private decimal _amount = 0;

  /// <summary>
  ///   The key used for encrypting the transaction data.
  /// </summary>
  private readonly byte[] key = Crypt.generateSalt();
  private readonly byte[] iv = Crypt.generateSalt(16);

  20 references | 17/17 passing
  public Transaction(IAccount ownAccount, IAccount destinationAccount, decimal amount)
  {
    _ownAccount = ownAccount;
    _destinationAccount = destinationAccount;
    _amount = amount;
  }
```

Transaction Flow

I. source and destination accounts are supplied,
II. amount is stored,
III. **SecureSend()** encrypts transaction details and stores them,
IV. **SecureReceive()** decrypts and validates them,
V. destination account is credited.

Figure 5 depicts the usual transaction flow for the Secure Banking App from the UI through to the Accounts classes

Figure 5: Transaction sequence diagram



2.3.3 Authentication Layer

*This layer is where the a*uthentication and security system is housed. It is the central focus of the Secure Banking App and mainly comprises the login subsystem and cryptography subsystem, Crypt.

The Login subsystem:

The login subsystem includes ILoginBasic, ILoginAdvanced, LoginBase, AdminLogin, and UserLogin. The LoginBase centralizes password hashing, sign-in state, password change, and salt management.

The UML flow for the login subsystem in Figure 6 depicts how, for instance the AdminLogin class implements both ILoginBasic and ILoginAdvanced interfaces and derives from the abstract LoginBase class.

The LoginBase class, being an abstract class, offers the possibility for its UserRole() method to be implemented differently for both AdminLogin and UserLogin. This inheritance hierarchy is a bit different for the UserLogin class which only implements the ILoginBasic interface and derives also from LoginBase.

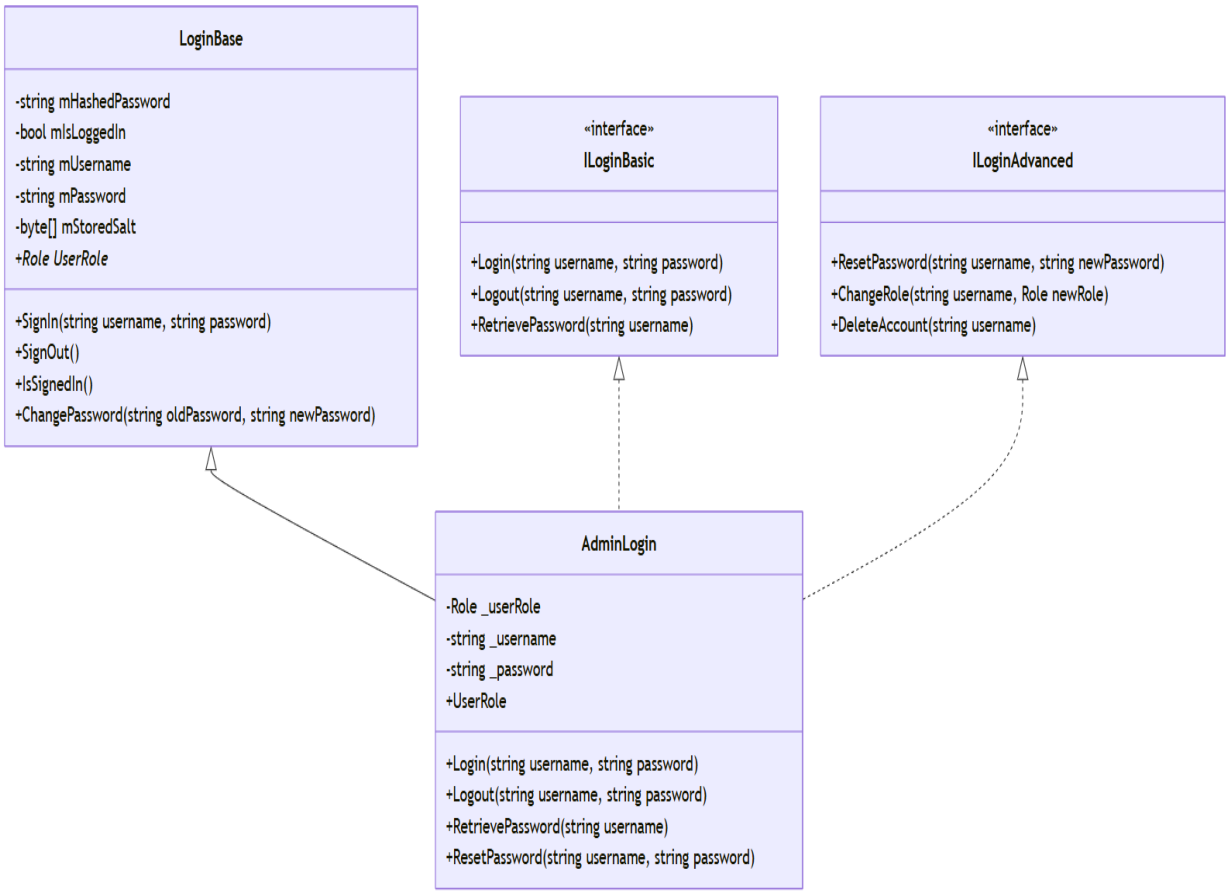
The Crypt:

Still part of the security layer, Crypt is built as a static utility class and provides the following cyber-security features:

- SHA-256 password hashing,
- password verification,
- random salt generation,
- AES encryption/decryption,
- HTML-based credential persistence.

This layer plays a key role in the Secure Banking App, whose focus is to show case the use of cyber security for protecting the transactions. The Crypt class is ultimately a utility service used by both login and transaction logic. Refer to *Code Snippet* 3.

Figure 6: UML diagram for the login subsystem



2.3.4 Persistence Layer

The persistence abstraction is IDatabase and offers two implementations as follows:

OfflineDatabase:

This is an in-memory singleton repository. It supports account storage but does not implement encrypted data methods at the moment. It uses the singleton design pattern and exposes only the interface defined methods in Code Snippet 3. The UML in Figure 7 also shows this class's components

SqlliteDatabase:

This is the main persistent implementation. It is considered as the main persistent layer of the Secure Banking App because it employs data encryption and decryption

when storing and retrieving accounts transactions. The implementation follows the singleton design pattern to get rid of the possibility of creating more than a single object per storage and retrieval cycle, which could ultimately lead to data loss.

The class does the following in summary:

- uses SQLite,
- stores accounts as serialized JSON,
- stores encrypted transaction payloads,
- uses singleton access,
- supports disposal and reconnection.

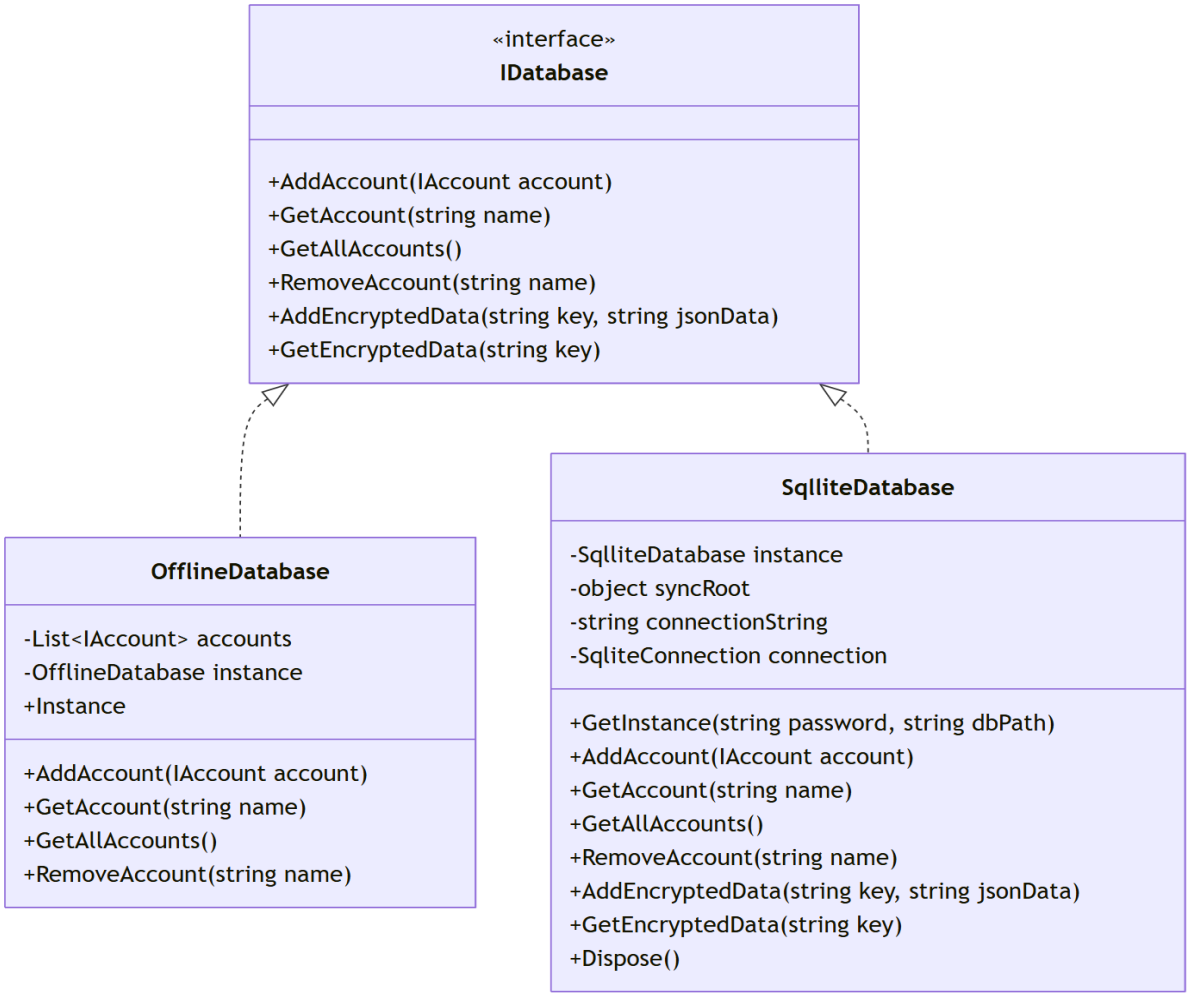
Code Snippet 3: The IDatabase interface and the Crypt class

```
using SecureBankingApp.Accounts;

namespace SecureBankingApp.Db
{
    8 references
    public interface IDatabase
    {
        12 references | 6/6 passing
        void AddAccount(IAccount account);
        7 references | 4/4 passing
        IAccount GetAccount(string name);
        4 references | 2/2 passing
        IEnumerable<IAccount> GetAllAccounts();
        4 references | 2/2 passing
        void RemoveAccount(string name);
        5 references
        void AddEncryptedData(string key, string jsonData);
        5 references
        string GetEncryptedData(string key);
    }
}
```

```
public class Crypt
{
    5 references | 3/3 passing
    public static string hashPassword(string password, byte[] salt)
    {
        var sha256 = SHA256.Create();
        var saltedPassword = Encoding.UTF8.GetBytes(password).Concat(sa
        var hash = sha256.ComputeHash(saltedPassword);
        return Convert.ToBase64String(hash);
    }

    5 references | 2/2 passing
    public static bool verifyPassword(string password, string storedH
    {
        var hash = hashPassword(password, salt);

        if (hash.Length != storedHash.Length)
            return false;

        int result = 0;
        for (int i = 0; i < hash.Length; i++)
        {
            result |= hash[i] ^ storedHash[i];
        }

        return result == 0;
    }

    6 references | 3/3 passing
    public static byte[] generateSalt(int size = 32)
    {
        byte[] salt = new byte[size];
        RandomNumberGenerator.Fill(salt);
        return salt;
    }

    1 reference
    public static string encryptData(string plainText, byte[] key, by
    {
        // Use the Advanced Encryption Standard (AES) for encryption
```

No issues found

Figure 7: UML diagram for the Persistent layer



2.4 Main Application Interaction Flow

The main user flow is:

a. Application starts in MainWindow.

b. User signs up or logs in.

c. MainWindow validates credentials.

d. Dashboard is loaded.

e. User may create savings/current accounts.

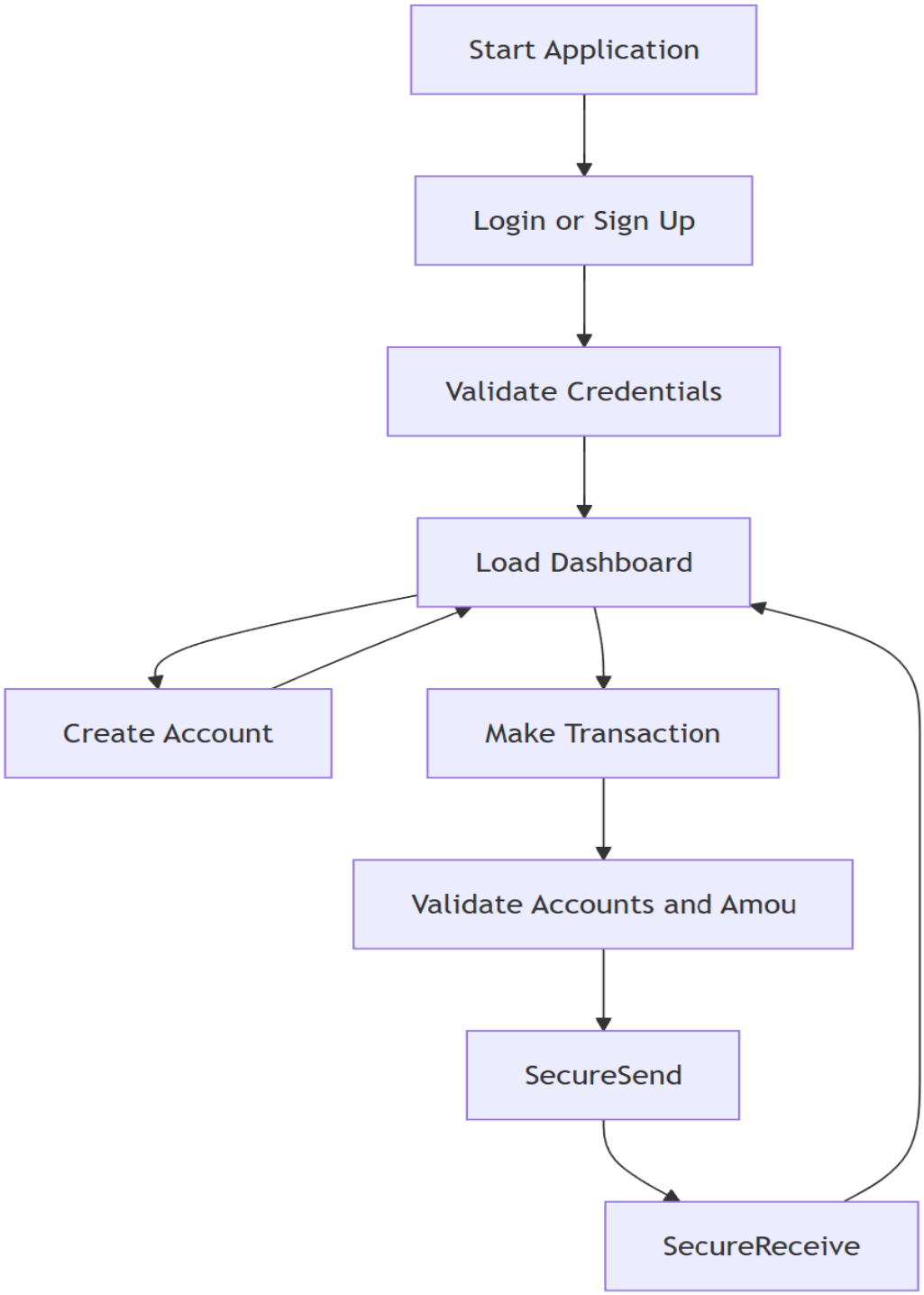
f. Users may initiate a transaction.

g. Transaction page raises events.

h. MainWindow performs transaction logic.

i. Database is updated.

Figure 8: the flow diagram for the Secure Banking App



2.5 Summary of Design Patterns

Several design patterns have been employed in the design of the Secure Banking App.

The Singleton Design Pattern

This design pattern is used in the implementation of the database classes as seen in OfflineDatabase where only the instance, OfflineDatabase.Instance is exposed. It is also used in the SqlliteDatabase as follows: **SqlliteDatabase.GetInstance(...)**

Purpose:

The purpose is to ensure one shared database instance.

The Factory Design pattern

This pattern is shown in the Factory AccountFactory class referenced through **AccountFactory.CreateAccount(...)** and **AccountFactory.CreateLogin<T>(...)**.

Purpose:

The purpose of the adoption of this pattern is to:

- centralize object creation,
- reduce coupling between UI and concrete classes.

Template Method / Inheritance

This pattern is used in AccountsBase and LoginBase. The instantiation of this class is however planned to follow the Strategy pattern, when more account types are added

Purpose:

The purpose of this pattern in the design includes:

- define shared behavior in base classes,
- let derived classes customize specific operations.

Observer/Event-driven pattern

This pattern is used heavily in WPF pages for button click events, custom events from pages to MainWindow.

Purpose:

> The purpose for the adoption of this pattern is to decouple UI controls from navigation logic.

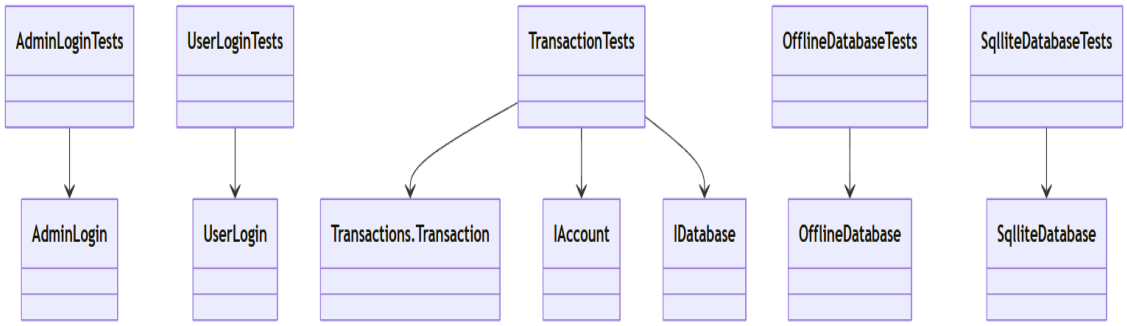
2.6 Unit Tests

To routinely test the Secure Banking App, the following Test stack are being used: MSTest.TestFramework, MSTest.TestAdapter, Moq and Microsoft.NET.Test.Sdk

The current tests cover areas such as the login behavior (AdminLogin and UserLogin), and the transaction behavior.

The database behavior comprising the OfflineDatabase and the SqlliteDatabase are tested and other behaviors like the positive-path, some negative-path, exception, basic concurrency checks and persistence CRUD (Create, Read, Update, and Delete). A full detail of the tests is contained in a separate document for want of space.

Figure 9: UML diagram showing the test classes interaction with the business logic



2.6.1 Test Summary

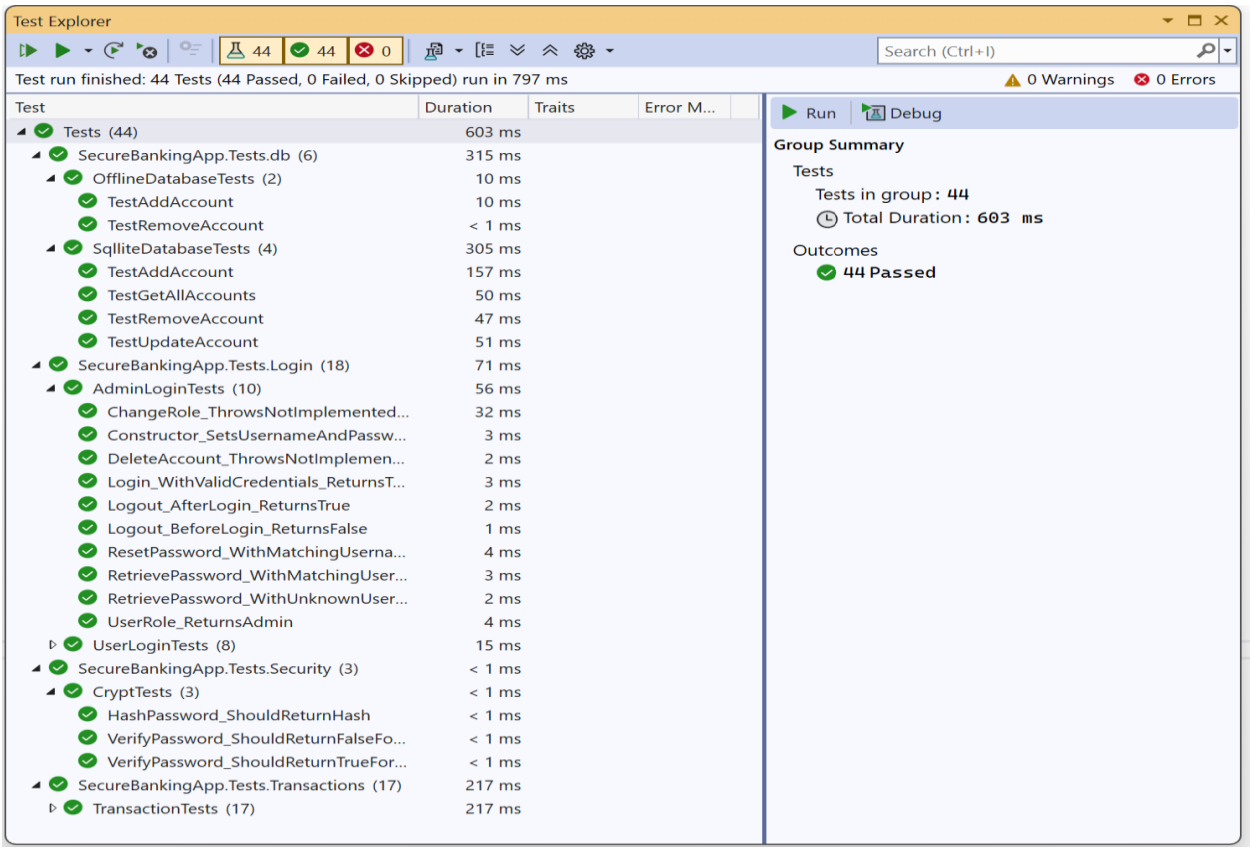
The full summary of test is submitted alongside this report as a spreadsheet. However, Figure 10 provides evidence of all the passing tests, proving a 100% pass rate for a total of 44

tests. The test currently does not cover the following WPF UI pages and bindings, MainWindow workflows, Crypt utility methods, factory classes as well as sign-up and dashboard page event behavior Table 1 below shows a few of the test cases for the transaction class

Table 1: The test classes and methods

Test	File	Tests covered
TransactionTests	Tests\Tests\TransactionTests.cs	1. Transfer_IsThreadSafe_WithMultipleThreads 2. Save_IsThreadSafe_WithMultipleThreads 3. Transfer_IsThreadSafe_WithConcurrentDifferentTransactions 4. Secure transaction tests 5. Secure_Send_and_Receive_WithValidData_CallsSecureSend 6. Secure_Send_and_Receive_WithValidData_CallsSecureReceive

Figure 10: Test Results from Visual studio Test Explorer



3 Academic Skill

Secure Banking App's development process brought along with it an enriching experience and helped develop and strengthen skills in the areas discussed under Key Learnings in section 3.1.

3.1 Key Learnings during Secure Banking App Development

Following the Gibbs Reflective Cycle (2003) [6] the important learnings during the project are discussed under the following headings:

Description:

The **SecureBankingApp** project involved designing and implementing a desktop banking system using **WPF**, **.NET 8**, and **C#**. The application included user registration, login, account creation, dashboard display, transaction handling, database persistence with **SQLite**, encryption utilities, and automated tests. The project combined user interface development, object-oriented design, persistence, security, and testing in one system.

Feelings:

Working through the project developed confidence in building a complete software application from multiple components rather than isolated classes. Coming from a hardware engineering background, building complete maintainable and safe embedded software has always been a challenge, until this project brought a boost of confidence, or maybe a great relief.

There were also particular moments of amazement such as in the case where deserialization of Json files became a problem in retrieving stored IAccount information and was after long hours of debugging eventually overcome. Yeah!!!

Evaluation:

A major positive learning outcome was the ability to apply object-oriented programming in a realistic context. Interfaces such as IAccount and IDatabase, together with base classes like AccountsBase and LoginBase, showed how abstraction and inheritance can support extensibility.

Another positive outcome was learning how different layers of an application interact. The project connected **WPF** pages, domain logic, persistence, and security utilities into one working system.

Security was also one of the main achievements, whose implementation triggered learning key points like hashing, salts, and AES encryption.

A critical post development assessment/review revealed some weaknesses in the architectural separation as much of the application flow is concentrated in MainWindow.xaml.cs, which shows that although the system works, the tight coupling in this class does not support maintainability, an area needing further enhancement.

Analysis:

The project provided several important technical and academic skills through independent research in the following areas:

Object-oriented Design

A key learning was how to model a banking system using interfaces and inheritance. IAccount defines a contract for account behavior, while SavingsAccount and CurrentAccount implement specialized rules. This reinforced understanding of abstraction, polymorphism, and reuse.

Event-driven Desktop App Development

The WPF pages and controls demonstrated how desktop applications respond to user actions through events. This helped build understanding of navigation, UI state changes, and communication between views and controller-like classes.

Persistence and Serialization

Using SqlliteDatabase showed how application data can be stored and retrieved persistently. The use of JSON serialization [3] for account objects also demonstrated how runtime objects can be mapped into a database-friendly format.

Security Concepts

The Crypt class and secure transaction flow introduced practical security concepts such as hashing, salt generation, encryption, and decryption. The Transaction class showed how transaction data can be protected before storage. At the same time, the project highlighted that using security APIs is not enough on its own; secure design decisions are equally important.

Concurrency and Thread Safety

The transaction logic showed the importance of synchronization when multiple operations may affect shared financial state. The use of ordered locking in Transaction.Transfer() was an important learning in avoiding deadlocks and preserving consistency.

Testing and Verification

The test project demonstrated how to validate behavior using MSTest [4] and Moq [5]. This reinforced the importance of testing not only successful outcomes but also exceptions, concurrency behavior, and persistence logic.

Conclusion:

The main learning from the project is that building a complete software system requires more than writing working code. It requires careful design, separation of concerns, testing, persistence strategy, and security awareness. The project strengthened practical skills in:

- object-oriented programming
- WPF application development
- database integration
- encryption and hashing concepts
- automated testing
- concurrency handling

It also showed that professional-quality software depends on stronger architecture, clearer validation, and more secure handling of sensitive data.

Action Plan

Based on this reflection, the following development measures are in the future:

- learn and improve architectural design by adopting **MVVM** [1] instead of relying heavily on code-behind
- learn to further separate concerns by moving business logic out of MainWindow.xaml.cs into dedicated services
- strengthen security by removing plain-text password retrieval and replacing HTML credential storage
- improve persistence by storing all critical account data consistently in the database
- expand test coverage to include Crypt, UI workflows, and more edge cases
- deepen understanding of secure software engineering and desktop application patterns

3.2 Summary of e-portfolio

The e-portfolio located at https://mowoh.github.io/.me/ details the achievements from the Advanced Object-Oriented Design and Programming January 2026. From the first unit to the last, focus has been on reviewing the basic OOP principles such as Inheritance, Polymorphism, Abstraction, and Encapsulation, building on it to further reestablish the more advanced topics such as the SOLID principle, upon which secure and modular applications are built.

These principles are greatly applied in the Secure Banking App development, for example the AccountBase encapsulation of accounts' metadata, the use of interfaces in IAccount and ILogin as well as the separation of concerns with Accounts, Crypt, Transactions, Login classes having their specific responsibilities, are all influenced by these topics

The Design patterns ranging from creational (Factory, Singleton) to behavioral (Observer) learned well as module were also applied, for example in the Offline and Online databases,

that are pure singletons, the accounts and login factory class implementing the factory pattern as well as the WPF GUI, that uses the event driven observers.

The success of this project consolidates the achievement of the key learning objectives of each of the units contained in the e-portfolio.

Much more advanced topics like Test driven development, Concurrency and Parallelism in Object-Oriented Design, Object-Oriented Software Architecture, Inversion of control, also influenced the design decisions during the development of the project

4 Professional Development Plan

This section summarizes the key professional skills initiated/strengthened during the project and asses the current level and the future development plan

Table 2: The PDP

Skill	Current Level	Expected Level	Time frame	Action Plan
Mastery of languages such as Python, C/C++/C#	Advanced	Expert	2 years	Accept more software related tasks and responsibilities at work, read books on Algorithms and Data structures
Software architecture and systems design	Beginner	Intermediate	6 months	Extending development of the Secure Banking App, enroll on online courses on software architecture and systems design
Database Management	Intermediate	Advanced	3 months	Extend Secure Banking App to use SQL, study books on SQL
Cyber security and cloud computing	Beginner	Advanced	3 years	Enrolling on online trainings
Embedded software Design	Advanced	Expert	2 years	Accept more hardware interfacing software tasks at work
Project management	Nill	Beginner	6 months	Enroll and obtain a certification from an instructor led course online
Time Management	Intermediate	Advanced	3 months	Research different approaches to self-organization and task prioritization

5 Conclusion

The **SecureBankingApp** project demonstrates the successful development of a functional banking application using **WPF**, **C#**, **.NET 8**, and **SQLite**, with focus on security.

The project brings together several important areas of software engineering, including object-oriented design, event-driven user interface development, database persistence, encryption using the AES library, and automated testing. Through features such as user registration, login, account creation, dashboard management, and secure transaction handling, the system shows a clear understanding of how different software components can be integrated into a working application.

A major strength of the project is its use of abstraction and modular design. Interfaces such as IAccount and IDatabase, together with concrete implementations like SavingsAccount, CurrentAccount, OfflineDatabase, and SqlliteDatabase, show an ability to structure code in a reusable and extensible way. The inclusion of a test project further strengthens the work by demonstrating awareness of software quality insurance through test driven design approach

az

References

1. Model-View-ViewModel - .NET | Microsoft Learn
2. Advanced Encryption Standard - Wikipedia
3. How to serialize JSON in C# - .NET | Microsoft Learn
4. Get started with C# and MSTest - .NET | Microsoft Learn
5. devlooped/moq: The most popular and friendly mocking framework for .NET
6. ReflectiveCycleGibbs.pdf
7. lock_ordering - Rust
8. Trujillo, A.E., Madni, A.M. (2023). MBSE Methods for Inheritance and Design Reuse. In: Madni, A.M., Augustine, N., Sievers, M. (eds) Handbook of Model-Based Systems Engineering. Springer, Cham
9. What is Windows Presentation Foundation - WPF | Microsoft Learn
10. Overview - Microsoft.Data.Sqlite | Microsoft Learn
11. Unified Modeling Language – Wikipedia